

Instituto Politécnico Nacional
Escuela Superior de Cómputo

Informe Técnico del Proyecto Web

Music Recommender API

Sistema backend para recomendación musical

Java, Maven, Spring Boot, Supabase, Render, Docker y GitHub

Alumnos: Cortés Avilez Yoshua, Gutiérrez López Samuel, Luciano Alvarado Ana Cristina, Pérez Montané Sergio Patricio

Profesor: M. en C. José Asunción Enríquez Zárte

Materia: Desarrollo Web

Grupo: 4BM2

Fecha: 1 de junio de 2026

Ciudad de México, México

Índice

1. Introducción	3
2. Arquitectura del Sistema	3
2.1. Descripción general	3
2.2. Componentes principales	3
2.3. Diagrama general de arquitectura	4
3. Código Fuente del Proyecto	4
3.1. Repositorio GitHub	4
4. Despliegue de la API	4
4.1. Render	4
4.2. URL pública de la API	5
4.3. Flujo de despliegue	5
4.4. Evidencia del despliegue en Render	5
5. Base de Datos	6
5.1. Supabase	6
5.2. URL del proyecto en Supabase	6
5.3. Diagrama de base de datos	6
5.4. Tablas de la base de datos	7
5.5. Evidencia del proyecto en Supabase	8
6. Script SQL Actualizado	9
6.1. Contenido requerido	9
6.2. Archivo entregado	9
6.3. Evidencia de ejecución del script en Supabase	10
7. Swagger UI	10
7.1. URL de Swagger UI	11
8. Envío de Correos	11
8.1. Funcionamiento general	11
9. Evidencia de Subida de Archivos	11
9.1. Funcionamiento general	12
10. Credenciales y Acceso a la Base de Datos	12
10.1. Datos de conexión	12
10.2. Evidencia de credenciales de conexión	13
11. Usuario de Prueba	13
12. Conclusiones	13

13. Anexos	14
13.1. Repositorio GitHub	14
13.2. Swagger UI y API desplegada	14
13.3. Supabase	14

1. Introducción

El presente informe describe la arquitectura, configuración, despliegue y funcionamiento general del proyecto web denominado *Music Recommender API*. El sistema consiste en una API desarrollada como backend, cuyo objetivo es gestionar operaciones relacionadas con una aplicación de recomendación musical.

El proyecto fue desarrollado utilizando Java, Maven y Spring Boot. Para la administración de la base de datos se utilizó Supabase, el cual proporciona una instancia de PostgreSQL en la nube. El código fuente fue almacenado y gestionado mediante GitHub, mientras que el despliegue del servicio se realizó en Render. Asimismo, el proyecto cuenta con documentación de endpoints mediante Swagger UI.

2. Arquitectura del Sistema

2.1. Descripción general

El sistema utiliza una arquitectura basada en servicios web. El backend se encarga de recibir las solicitudes HTTP, procesarlas mediante controladores, aplicar la lógica de negocio correspondiente y comunicarse con la base de datos alojada en Supabase.

El flujo general del sistema puede describirse de la siguiente manera:

1. El usuario o cliente realiza una solicitud HTTP hacia la API desplegada en Render.
2. Render ejecuta la aplicación backend desarrollada con Spring Boot.
3. La API procesa la solicitud mediante sus controladores, servicios y repositorios.
4. El backend se comunica con Supabase para consultar, insertar, actualizar o eliminar información.
5. La respuesta se devuelve al cliente en formato JSON.

2.2. Componentes principales

Componente	Descripción
Java	Lenguaje utilizado para el desarrollo del backend.
Maven	Herramienta utilizada para la gestión de dependencias y construcción del proyecto.
Spring Boot	Framework utilizado para crear la API REST.
Supabase	Plataforma utilizada para alojar la base de datos PostgreSQL.
PostgreSQL	Sistema gestor de base de datos relacional utilizado por Supabase.
GitHub	Plataforma utilizada para el control de versiones y almacenamiento del código fuente.
Render	Plataforma utilizada para desplegar públicamente la API.

Dockerfile	Archivo utilizado para definir el entorno de construcción y ejecución del backend.
Swagger UI	Interfaz utilizada para documentar y probar los endpoints de la API.

2.3. Diagrama general de arquitectura

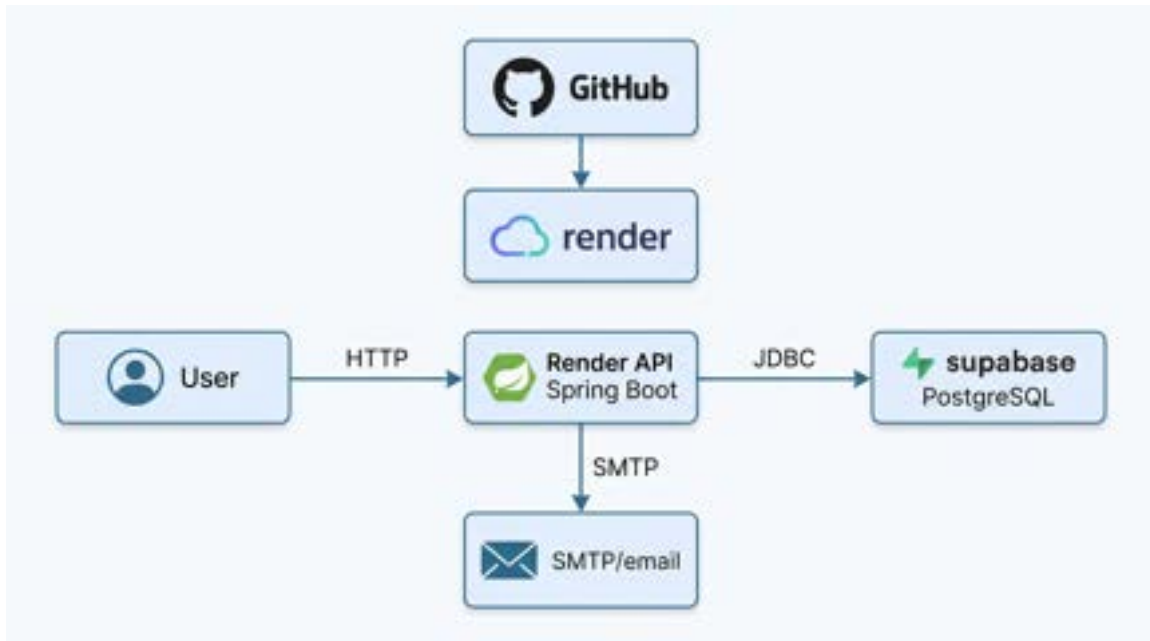


Figura 1: Diagrama general de arquitectura del sistema

3. Código Fuente del Proyecto

El código fuente del proyecto se encuentra alojado en GitHub, donde se administra el control de versiones de la aplicación backend.

3.1. Repositorio GitHub

<https://github.com/SerPatPM/music-recommender.git>

El repositorio contiene la estructura principal del proyecto, incluyendo el código fuente, archivo `pom.xml`, configuración de Maven, archivos compilados y el archivo `Dockerfile` utilizado para el despliegue.

4. Despliegue de la API

4.1. Render

La API fue desplegada en Render. Esta plataforma permite ejecutar el backend en la nube y exponerlo mediante una URL pública.

4.2. URL pública de la API

<https://music-recommender-mjkm.onrender.com/swagger-ui/index.html#/>

4.3. Flujo de despliegue

El flujo de despliegue utilizado en el proyecto es el siguiente:

1. Se realizan modificaciones en el código fuente del backend.
2. Los cambios se suben al repositorio de GitHub.
3. Render detecta los cambios realizados en el repositorio.
4. Render ejecuta el proceso de construcción definido para el proyecto.
5. La API queda actualizada y disponible públicamente.

4.4. Evidencia del despliegue en Render

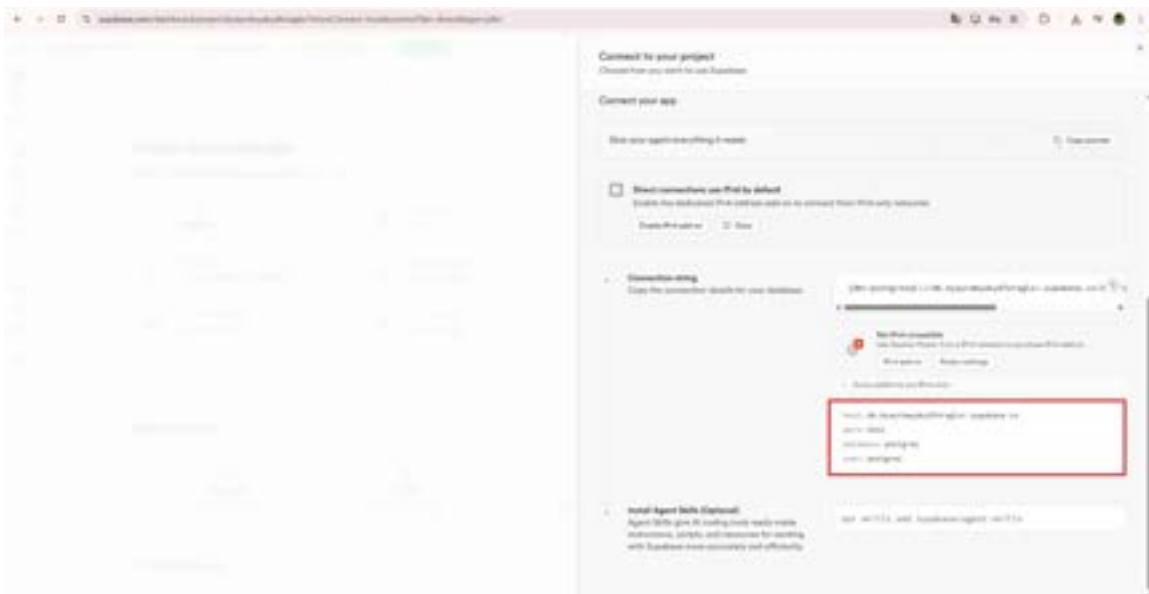


Figura 2: Creación de un nuevo Web Service en Render

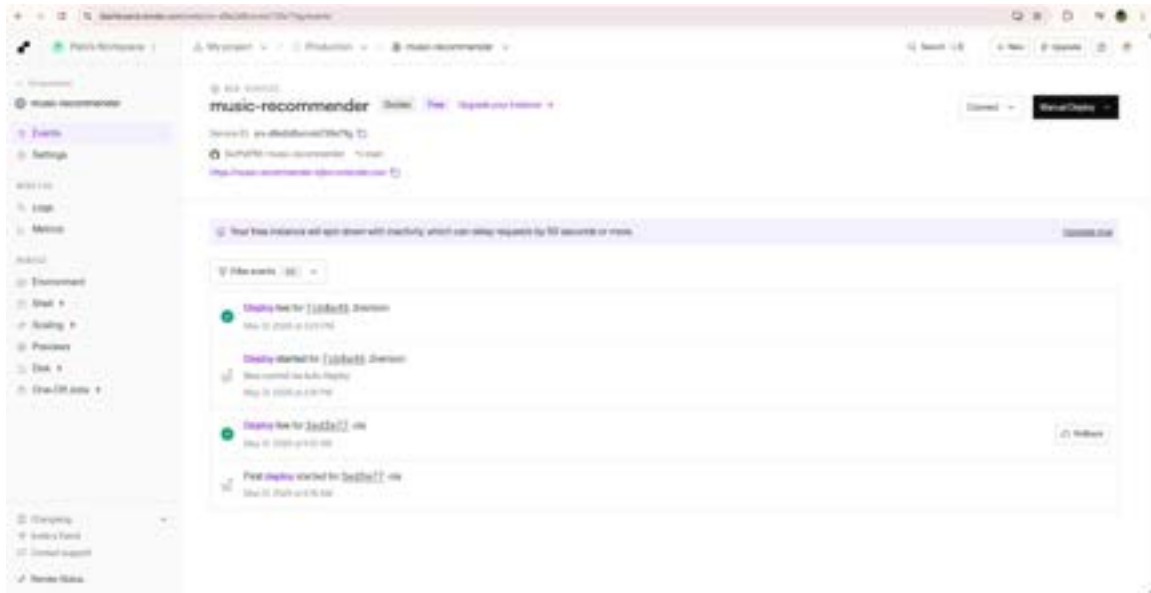


Figura 3: Historial de eventos y deploys exitosos en Render

5. Base de Datos

5.1. Supabase

La base de datos del sistema se encuentra alojada en Supabase. Esta plataforma utiliza PostgreSQL y permite administrar las tablas, relaciones y registros del sistema desde una interfaz web.

5.2. URL del proyecto en Supabase

<https://supabase.com/dashboard/project/byayrdwyakydforaglxr>

5.3. Diagrama de base de datos

En esta sección se debe colocar el diagrama entidad-relación final de la base de datos.

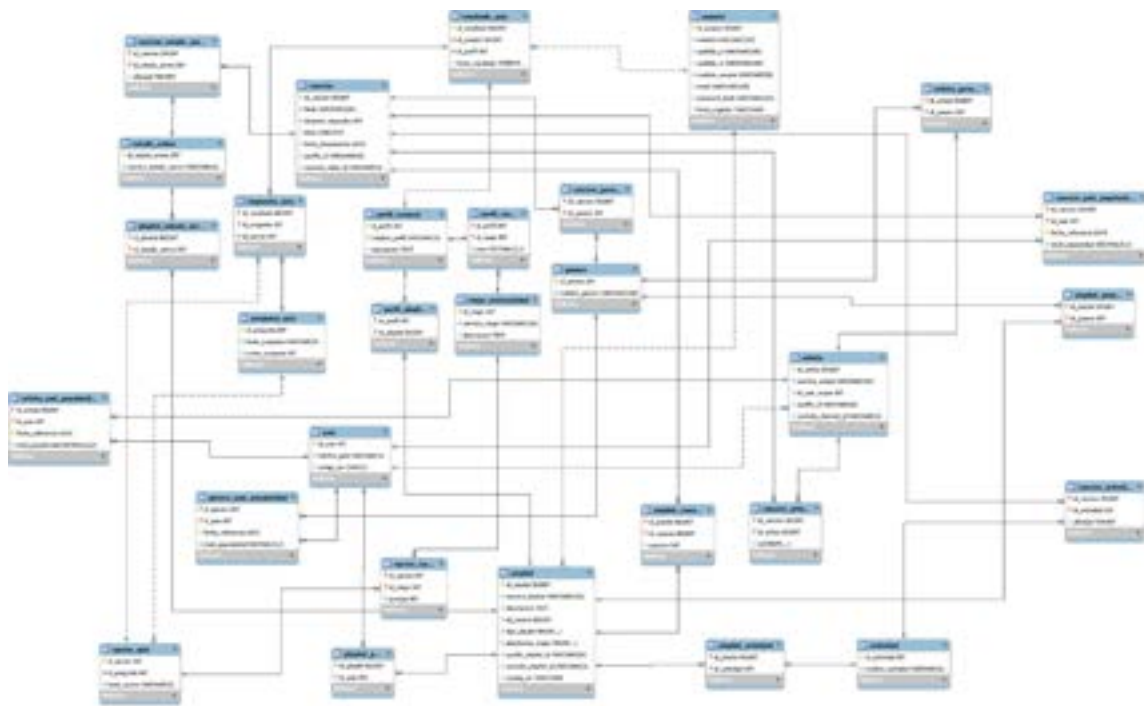


Figura 4: Diagrama entidad-relación de la base de datos

5.4. Tablas de la base de datos

#	Tabla	Descripción
Usuarios y Quiz		
1	usuario	Usuarios registrados del sistema
2	resultado_quiz	Resultados de los quizzes realizados
3	respuesta_quiz	Respuestas individuales del quiz
4	pregunta_quiz	Preguntas del cuestionario
5	opcion_quiz	Opciones de respuesta por pregunta
6	opcion_rasgo	Relación entre opciones del quiz y rasgos de personalidad
Música (Entidades principales)		
7	cancion	Canciones del catálogo
8	artista	Artistas musicales
9	playlist	Playlists creadas o importadas
10	genero	Géneros musicales
11	pais	Países (para popularidad y origen)
Perfil Musical y Personalidad		
12	perfil_musical	Perfiles de gusto musical
13	perfil_rasgo	Relación perfil ↔ rasgo (con peso)
14	perfil_playlist	Relación perfil ↔ playlist
15	rasgo_personalidad	Rasgos de personalidad del usuario
Estados de Ánimo y Actividades		
16	estado_animo	Estados de ánimo posibles
17	actividad	Actividades y contextos

Continúa en la siguiente página...

#	Tabla	Descripción
Tablas de Relación N:M		
18	cancion_estado_animo	canción ↔ estado de ánimo (con afinidad)
19	playlist_estado_animo	playlist ↔ estado de ánimo
20	cancion_genero	canción ↔ género
21	artista_genero	artista ↔ género
22	playlist_genero	playlist ↔ género
23	cancion_pais_popularidad	canción ↔ país (popularidad por fecha)
24	artista_pais_popularidad	artista ↔ país (popularidad por fecha)
25	genero_pais_popularidad	género ↔ país (popularidad por fecha)
26	playlist_cancion	playlist ↔ canción (con posición)
27	cancion_artista	canción ↔ artista (con rol)
28	cancion_actividad	canción ↔ actividad (con afinidad)
29	playlist_actividad	playlist ↔ actividad
30	playlist_pais	playlist ↔ país

5.5. Evidencia del proyecto en Supabase

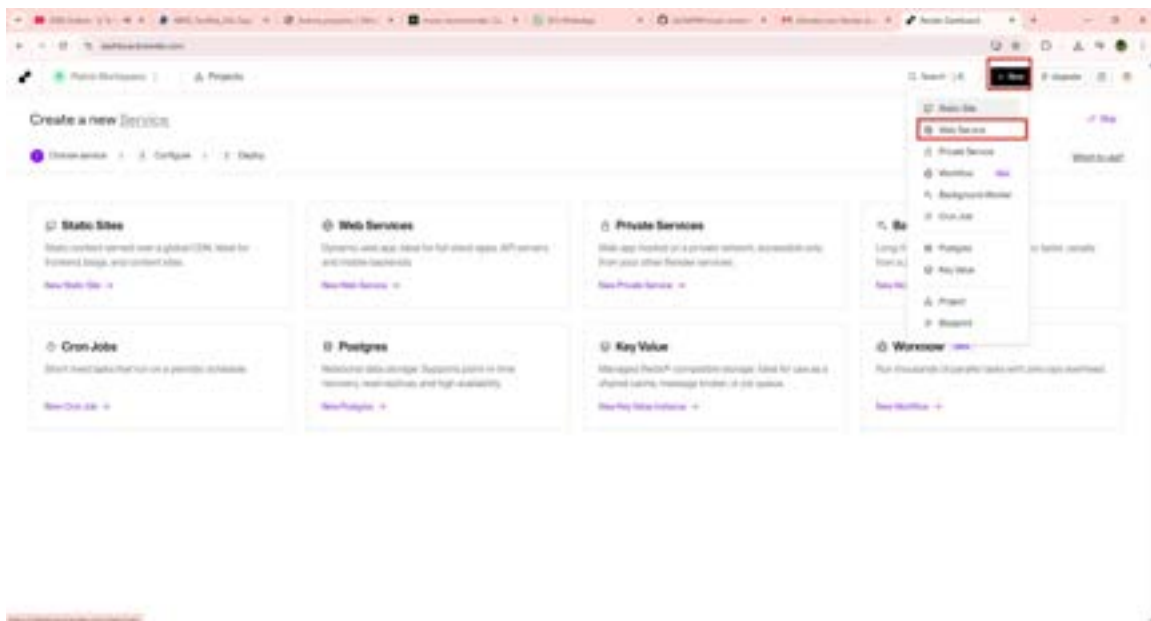


Figura 5: Configuración del proyecto music-recommender en Supabase

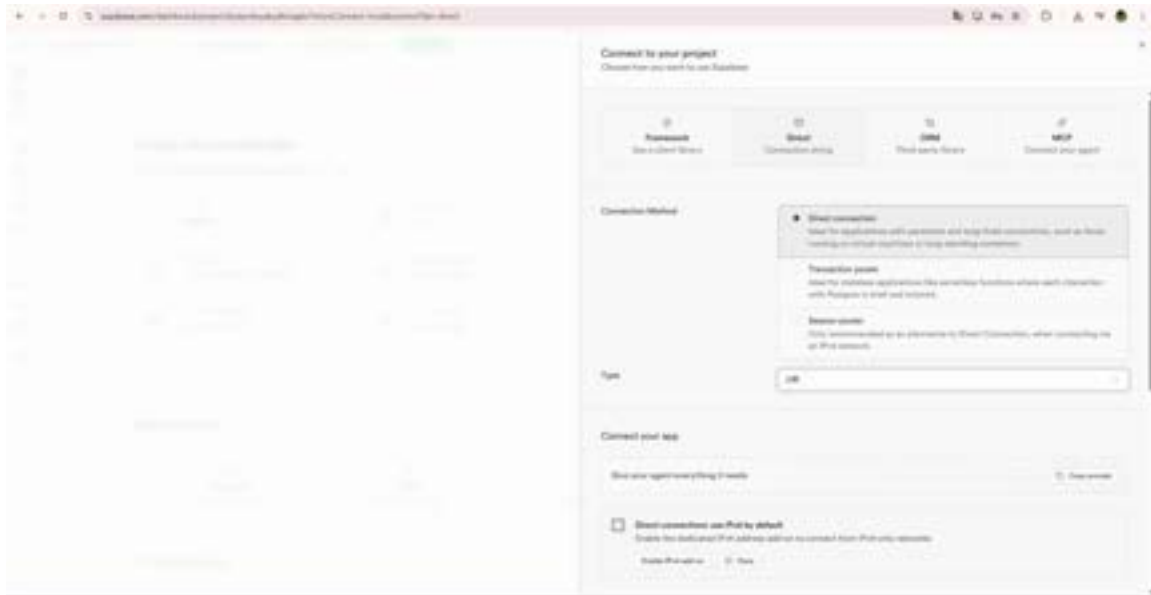


Figura 6: Opciones de conexión al proyecto en Supabase

6. Script SQL Actualizado

El archivo SQL actualizado debe incluir las sentencias necesarias para la creación de tablas, restricciones, relaciones e inserción de datos de prueba.

6.1. Contenido requerido

- Sentencias DDL para la creación de tablas.
- Llaves primarias y llaves foráneas.
- Restricciones de integridad.
- Sentencias DML para insertar datos de prueba.

6.2. Archivo entregado

El script será entregado como archivo independiente con el nombre:

`script_music_recommender.sql`

6.3. Evidencia de ejecución del script en Supabase

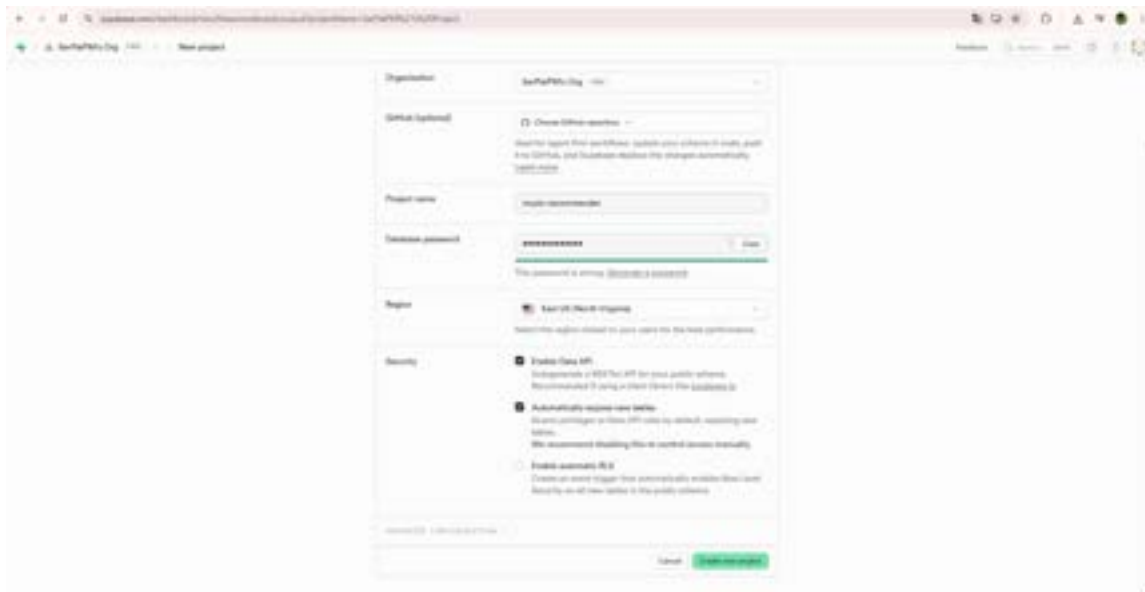


Figura 7: Ejecución del script SQL en el editor de Supabase

7. Swagger UI

La API cuenta con documentación mediante Swagger UI, lo cual permite visualizar y probar los endpoints disponibles desde el navegador.

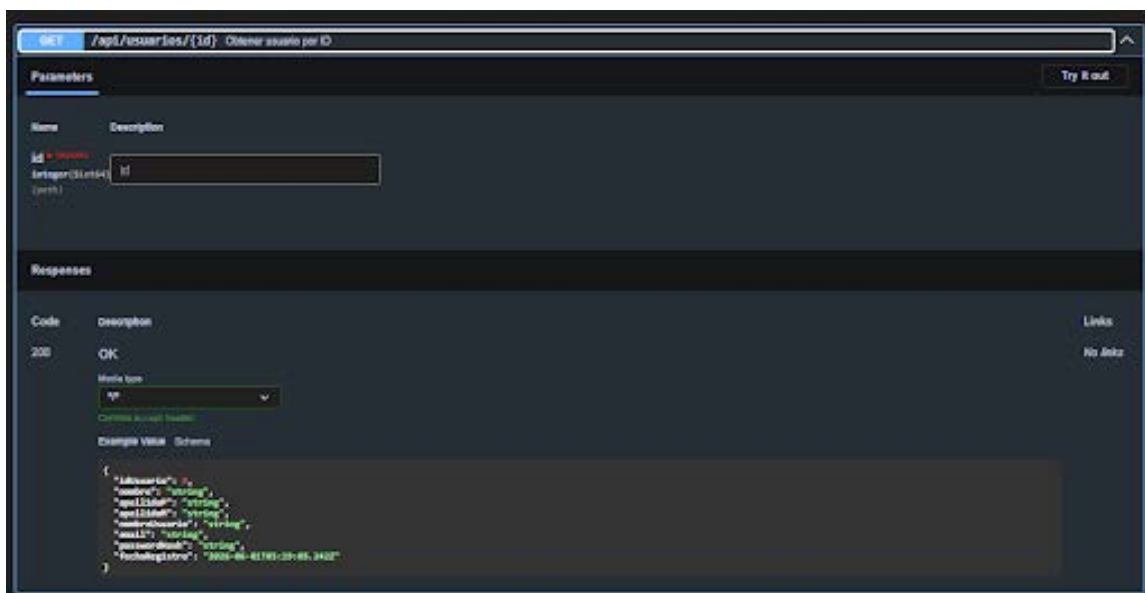


Figura 8: Prueba 1 swagger

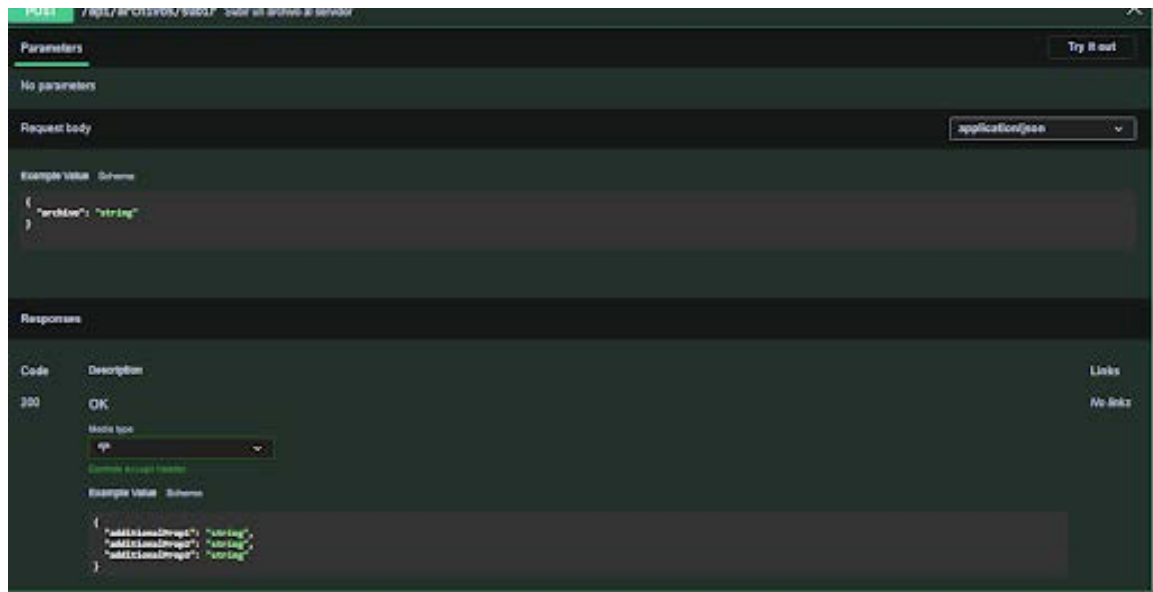


Figura 9: Prueba 2 swagger

7.1. URL de Swagger UI

<https://music-recommender-mjkm.onrender.com/swagger-ui/index.html#/>

8. Envío de Correos

El sistema cuenta con funcionalidad para el envío de correos electrónicos. Esta función permite notificar eventos o cambios relacionados con la aplicación o con los registros administrados por el sistema.

8.1. Funcionamiento general

El backend procesa la solicitud correspondiente, genera el contenido del mensaje y utiliza el servicio configurado para enviar el correo electrónico al destinatario definido.

9. Evidencia de Subida de Archivos

El sistema contempla la funcionalidad de carga de archivos mediante endpoints de la API. Esta característica permite recibir archivos desde el cliente y procesarlos desde el backend.

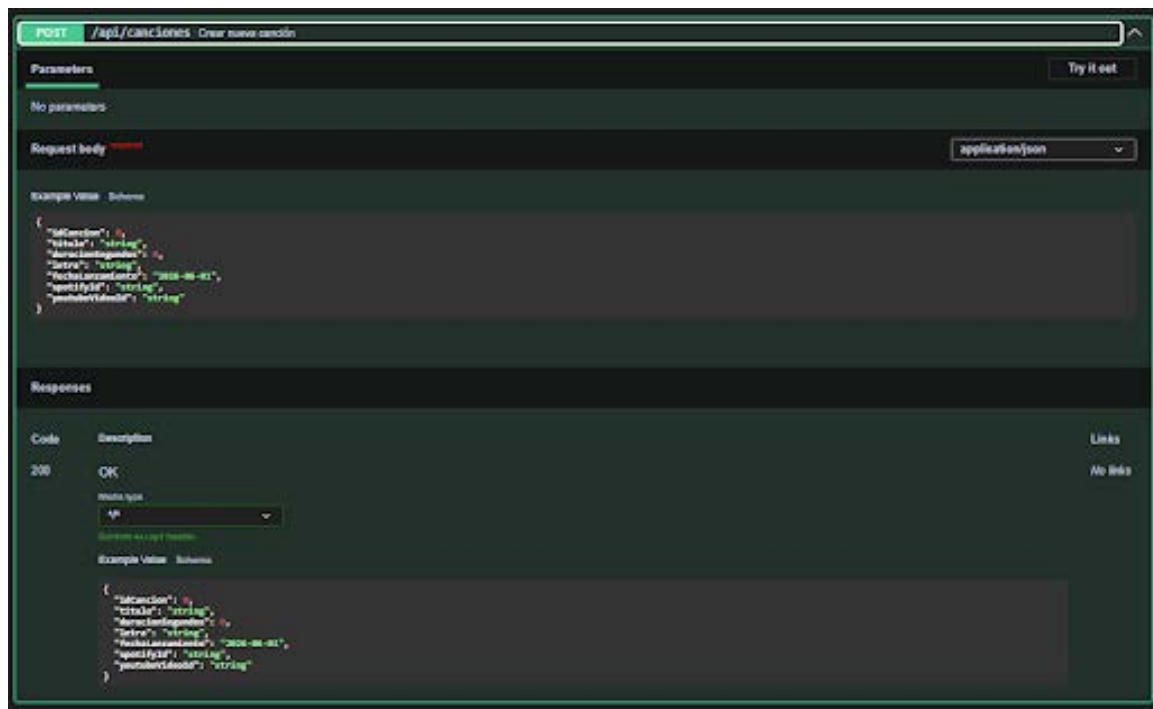


Figura 10: Prueba 2 swagger

9.1. Funcionamiento general

El usuario realiza una petición al endpoint correspondiente, adjuntando el archivo que se desea cargar. Posteriormente, la API procesa la solicitud y registra la información relacionada con el archivo.

10. Credenciales y Acceso a la Base de Datos

En esta sección se presentan los datos necesarios para validar la conexión a la instancia de base de datos utilizada por el sistema.

10.1. Datos de conexión

Elemento	Valor
Host	db.byayrdwyakydforaglxr.supabase.co
Puerto	5432
Usuario	postgres
Contraseña	Colocar contraseña correspondiente
Nombre de la base de datos	postgres
URL del proyecto	https://supabase.com/dashboard/project/byayrdwyakydforaglxr

Nota: Las credenciales se proporcionan únicamente para fines de validación académica del proyecto.

10.2. Evidencia de credenciales de conexión



Figura 11: Cadena de conexión JDBC y datos de acceso a la base de datos en Supabase

11. Usuario de Prueba

En caso de que la API requiera autenticación, se proporciona el siguiente usuario de prueba:

Campo	Valor
Usuario o correo	patriciopmontane05@gmail.com
Contraseña	Contraseña1234

Si la API no requiere autenticación, se deberá indicar que los endpoints pueden consultarse directamente desde Swagger UI.

12. Conclusiones

El desarrollo del proyecto permitió integrar distintas herramientas utilizadas actualmente en el despliegue de aplicaciones web. La API fue desarrollada con Java, Maven y Spring Boot, lo que permitió estructurar el backend mediante controladores, servicios y acceso a datos.

El uso de GitHub facilitó el control de versiones del código fuente, mientras que Render permitió desplegar la API en la nube y hacerla accesible mediante una URL pública. Por su parte, Supabase proporcionó una base de datos PostgreSQL administrada en la nube, permitiendo consultar y modificar registros de manera remota.

Finalmente, Swagger UI permitió documentar y probar los endpoints de la API, lo cual facilita la validación del funcionamiento del sistema. En conjunto, estas herramientas permitieron construir una solución backend funcional, documentada y desplegada en un entorno web.

13. Anexos

13.1. Repositorio GitHub

<https://github.com/SerPatPM/music-recommender.git>

13.2. Swagger UI y API desplegada

<https://music-recommender-mjkm.onrender.com/swagger-ui/index.html#/>

13.3. Supabase

<https://supabase.com/dashboard/project/byayrdwyakydforaglxr>